linalg_pedagopy

Plan de développement v5

Consolider l’architecture, puis ajouter déterminants,
valeurs propres et diagonalisation

Cédric Bohnert

19 octobre 2026 — branche `development`

Sommaire

1. Objet et résumé **3**

2. État des lieux (19 octobre 2026) **3**

2.1 Historique récent de la branche `development` . . . 3

2.2 Mesures . . . 3

2.3 Constats techniques . . . 4

3. Principes de conduite **4**

4. Phase 0 — Filet de sécurité **5**

4.1 Tâches . . . 5

4.2 Critères de fin . . . 5

5. Phase 1 — Moteur d'élimination réutilisable **5**

5.1 Responsabilités actuellement mélangées dans `_solve_under_condition` . . . 5

5.2 Conception proposée . . . 6

5.3 Étapes . . . 6

5.4 Risques . . . 7

6. Phase 2 — Alléger `Matrix` **7**

7. Phase 3 — Découpage du tuteur et suppression des `eval` **7**

7.1 Arborescence cible . . . 7

7.2 Tâches . . . 7

8. Phase 4 — Déterminant exact avec discussion de cas **8**

8.1 Fonctionnalités . . . 8

8.2 Rendu et locales . . . 9

8.3 Intégration . . . 9

8.4 Tests . . . 9

9. Phase 5 — Valeurs propres, espaces propres, diagonalisation **9**

9.1 Fonctionnalités . . . 9

9.2 Choix mathématiques à trancher . . . 10

9.3 Pédagogie . . . 10

9.4 Tests . . . 10

10. Phase 6 — Extensions ultérieures **11**

11. Calendrier et dépendances **11**

12. Risques et parades **12**

13. Décisions à prendre **12**

14. Définition de « terminé » (pour chaque étape) **12**

Annexe A — Point de reprise pour la prochaine séance **13**

Annexe B — Esquisse de l'API cible (version 3.1) **13**

1. Objet et résumé

Ce plan organise la suite du développement de `linalg_pedagopy` sur la branche `development`. Il part d'un constat : avant d'ajouter de grandes fonctionnalités mathématiques (déterminant, valeurs propres, diagonalisation), il faut **consolider l'architecture** pour que le nouveau code se range dans une structure saine et réutilise la logique existante au lieu de la dupliquer.

Le plan comporte **six phases**, chacune livrée en commits indépendants, avec la suite de tests verte avant et après :

Phase	Contenu	Nature	Effort estimé
0	Filet de sécurité : couverture, tests de caractérisation, intégration continue	outillage	1 à 2 séances
1	Extraction d'un **moteur d'élimination** depuis `GaussJordan`	refactorisation (sans changement de comportement)	2 à 3 séances
2	Allègement de `Matrix` (règle de délégation)	refactorisation	1 séance
3	Découpage du tuteur (`verifiers.py`, `catalog.py`) et suppression des `eval`	refactorisation + sécurité	2 séances
4	**Déterminant** exact avec discussion de cas	fonctionnalité	2 à 3 séances
5	**Valeurs propres, espaces propres, diagonalisation**	fonctionnalité	3 à 4 séances
6	Extensions ultérieures (exercices paramétrés, sous-espaces, géométrie, LU…)	feuille de route	—

Une *séance* désigne une demi-journée à une journée de travail. Les phases 0 à 3 ne changent **aucun comportement visible** : les sorties des carnets et de la documentation doivent rester identiques.

2. État des lieux (19 octobre 2026)

2.1 Historique récent de la branche `development`

Commit	Contenu
`0ed9dde`	Accents des fiches restaurés, diagnostic `row_overwrite`
`4ca4cbf`	Documentation alignée sur l'API réelle
`65a985d`	Exercices nommés (35 fiches, 8 thèmes), catalogue, corrections, documentation complète
(non commité)	Cahier de 20 exercices `exercices/`, correction de `GaussJordan(A, compute_inverse=True)`

2.2 Mesures

- **Code** : environ 11 350 lignes dans `src/linalg_pedagopy`.

- **Tests** : 71 fonctions de test (17 fichiers), toutes vertes ; la validation croisée des 35 fiches fait partie de la suite.
- **Documentation** : 9 guides Markdown + site HTML généré (`build_docs_site.py`), tutoriel, 4 carnets.

Les plus gros modules :

Module	Lignes	Observation
`tutor/verifiers.py`	1 071	9 vérificateurs et leurs utilitaires dans un seul fichier
`tutor/catalog.py`	918	catalogue, séance, rapport de validation et contrôle croisé mélangés
`objects/matrix.py`	777	56 méthodes ; délègue déjà `rref`, `rank`, `nullspace`, `inv`, `solve` à `GaussJordan`
`exercises/generators.py`	723	20 générateurs d'exercices jumeaux
`algorithms/gauss_jordan.py`	699	**`_solve_under_condition` fait à elle seule environ 290 lignes**

2.3 Constats techniques

1. **La logique d'élimination n'est pas réutilisable.** Le choix du pivot, l'interrogation de l'oracle de nullité, la séparation en cas (`_split_on_factors`) et l'enregistrement de la trace sont imbriqués dans `GaussJordan._solve_under_condition`. Le déterminant et le calcul des espaces propres ont besoin exactement de cette machinerie.
2. **`Matrix` grossit par ajout de méthodes.** Ajouter `det()`, `eigenvalues()`, `diagonalize()` directement dans `matrix.py` ferait dépasser 1 000 lignes.
3. **Le tuteur concentre deux gros fichiers.** Les vérificateurs, le contrôle croisé et la séance évoluent pour des raisons différentes ; les garder ensemble rend chaque modification risquée.
4. **Dix appels à `eval`** subsistent (`tutor/lab.py`, `hints.py`, `panel.py`, `verifiers.py`, `catalog.py`), tous avec `__builtins__` vide, mais un interpréteur fondé sur `ast` serait plus sûr et plus lisible.
5. **Outillage absent** : pas de mesure de couverture (`coverage` non installé), pas d'intégration continue, `requires-python >= 3.11` alors que le code tourne en 3.10.
6. **Manques mathématiques** : ni déterminant, ni polynôme caractéristique, ni valeurs propres, ni LU, ni sous-espaces comme objets.

3. Principes de conduite

- **Une étape = un commit** (ou une petite série), message en français au format `type(portée): résumé` (`refactor`, `feat`, `fix`, `test`, `docs`, `ci`).
- **Refactorisation pure dans les phases 1 à 3** : aucun changement de sortie. La référence est fixée en phase 0 par des tests de caractérisation (traces, discussions de cas, diagnostics du tuteur).
- **Compatibilité de l'API publique** : tout nom importable aujourd'hui depuis `linalg_pedagopy`, `linalg_pedagopy.tutor` ou `linalg_pedagopy.tuteur` le reste. Les anciens chemins internes déplacés sont conservés par réexportation pendant au moins une version.
- **Chaque fonctionnalité nouvelle arrive complète** : code, tests, entrées de locale `fr.yaml`, rendu (texte, LaTeX, Markdown, HTML), documentation (guide + référence API), au moins un exercice ou une fiche.
- **Le calcul reste exact et sur** $\mathbb{R}$; les cas indécidables passent par l'oracle et `CaseSplit`, jamais par un choix arbitraire.

4. Phase 0 — Filet de sécurité

Objectif : pouvoir refactoriser en sachant immédiatement si un comportement change.

4.1 Tâches

N°	Tâche	Détail
0.1	Mesurer la couverture	`pip install coverage` ; `coverage run -m pytest` ; rapport par module. Cible indicative : ≥ 85 % sur `algorithms/`, `core/`, `tutor/`.
0.2	Tests de caractérisation des traces	Pour 8 à 10 calculs représentatifs (systèmes unique / affine / incompatible, inverse, paramètre a, deux paramètres a, b, radicaux), figer `explain(level=1..3).to_text()` et `to_markdown()` dans des fichiers de référence (`tests/references/*.txt`).
0.3	Tests de caractérisation du tuteur	Pour chaque type de vérificateur, figer le diagnostic d'une réponse fausse typique et le message d'une réponse juste.
0.4	Tests des carnets	Script `tests/run_notebooks.py` exécutant les carnets générés (tutoriel, problèmes, cahier d'exercices + corrigé) et échouant sur la première erreur.
0.5	Intégration continue	`.github/workflows/tests.yml` : Python 3.10 à 3.13, `pytest`, `ruff`, `mypy --strict` (non bloquant au début), validation des 35 fiches, exécution des carnets.
0.6	Version de Python	Trancher : abaisser `requires-python` à `>=3.10` (le code y tourne) ou tester réellement à partir de 3.11. Recommandation : `>=3.10`.
0.7	Commiter le travail en attente	Cahier `exercices/` et correctif `compute_inverse` (commit `feat(exercices) + fix(gauss-jordan)`).

4.2 Critères de fin

- Rapport de couverture disponible, zones faibles listées dans `docs/dev/couverture.md`.
- Fichiers de référence générés et tests de caractérisation verts.
- Pipeline CI vert sur la branche.

5. Phase 1 — Moteur d'élimination réutilisable

Objectif : isoler dans un module dédié la machinerie commune à tous les calculs par élimination, puis reconstruire `GaussJordan` dessus **sans changer une ligne de ses sorties**.

5.1 Responsabilités actuellement mélangées dans `_solve_under_condition`

1. Construction de la matrice augmentée et simplification des coefficients.
2. Parcours colonne par colonne, recherche des candidats pivots et interrogation de l'oracle.

3. Bascule vers une discussion de cas quand aucun candidat n'est inconditionnellement non nul.
4. Choix du pivot selon la stratégie (`simplest`, `first`, `partial`).
5. Opérations élémentaires (échange, dilatation, transvection) et leur enregistrement dans la trace.
6. Test de compatibilité des lignes nulles, construction de l'inverse et de l'ensemble des solutions.

Les points 1 à 5 sont génériques ; le point 6 est propre à Gauss-Jordan.

5.2 Conception proposée

Nouveau module `algorithms/elimination.py` :

```
@dataclass
class EliminationState:
    grid: list[list[sp.Expr]]       # matrice de travail (éventuellement augmentée)
    condition: Condition            # hypothèses de la branche courante
    pivots: list[tuple[int, int]]   # (ligne, colonne) des pivots trouvés
    swaps: int                      # nombre d'échanges (signe du déterminant)
    scale: sp.Expr                  # produit des facteurs de dilatation appliqués

class EliminationEngine:
    def __init__(self, *, pivot_strategy, reduce_above: bool, normalize_pivots: bool,
                 split_col: int | None, max_depth: int, max_cases: int): ...
    def run(self, A_aug: Matrix, cond: Condition, depth: int,
            on_complete: Callable[[EliminationState], R]) -> R | CaseSplit[R]: ...
```

- `reduce_above=True, normalize_pivots=True` donne la forme échelonnée réduite (Gauss-Jordan).
- `reduce_above=False, normalize_pivots=False` donne une forme triangulaire **sans dilatation**, ce dont le déterminant a besoin (produit des pivots, signe des échanges).
- `on_complete` reçoit l'état final d'une branche et fabrique le résultat métier (`GJResult`, `DetResult`, ...). La séparation en cas (`_split_on_factors`) devient une méthode du moteur, qui rappelle `run` sur chaque branche.
- Les clés de trace (`gj.pivot_chosen`, `gj.row_add`...) restent identiques ; le moteur reçoit un préfixe (`gj`, `det`) pour que les nouvelles procédures aient leurs propres messages.

5.3 Étapes

N°	Étape	Contrôle
1.1	Extraire les fonctions pures (`_simplify_entry`, `_pivot_complexity_key`, choix du pivot) dans `elimination.py`	tests verts
1.2	Extraire les opérations élémentaires tracées (`swap`, `scale`, `add_multiple`)	traces identiques (phase 0)
1.3	Extraire la boucle d'élimination et `_split_on_factors` dans `EliminationEngine`	traces et `CaseSplit` identiques
1.4	Réécrire `GaussJordan._solve_under_condition` comme appel au moteur + `on_complete`	`gauss_jordan.py` < 400 lignes
1.5	Tests unitaires propres au moteur (mode triangulaire sans dilatation, comptage des échanges)	nouveaux tests verts

5.4 Risques

- **Régression subtile dans l'ordre des branches ou des étapes** : couverte par les tests de caractérisation de la phase 0.
- **Performance** : l'indirection ajoute un coût négligeable devant SymPy ; mesurer sur un 5×5 paramétré avant et après.

6. Phase 2 — Alléger `Matrix`

Objectif : fixer une règle simple avant d'ajouter `det()` et `eigenvalues()`.

Règle de délégation : `Matrix` contient la représentation, les opérations algébriques (`+`, `@`, transposée, blocs, accès) et des **méthodes de commodité d'une à trois lignes** qui délèguent à une procédure de `algorithms/`. Toute logique d'algorithme (boucle, discussion de cas, trace) vit dans `algorithms/`.

N°	Tâche
2.1	Remplacer le crochet `_require_gj` par un registre générique `_procedures` (`"gauss_jordan"`, `"determinant"`, `"eigen"`…) alimenté par `algorithms/__init__.py`, sans import circulaire.
2.2	Regrouper les méthodes de commodité dans une section clairement délimitée ; vérifier qu'aucune ne dépasse quelques lignes.
2.3	Déplacer la logique de `matmul(view=…)` (vues entries/rows/columns/outer et leur trace) dans `algorithms/products.py`.
2.4	Documenter la règle dans `docs/dev/architecture.md`.

Critère de fin : `matrix.py` sous 600 lignes, API inchangée, tests verts.

7. Phase 3 — Découpage du tuteur et suppression des `eval`

7.1 Arborescence cible

```
tutor/
  __init__.py            # API publique inchangée (réexportations)
  catalog.py             # chargement des fiches, catalog(), themes(), résolution des références
  session.py             # ExerciseSession, MarkdownText, TeacherSheet
  validation.py          # ValidationReport, validate_sheet, contrôle croisé
  safe_eval.py           # interpréteur d'expressions fondé sur ast
  verifiers/
    __init__.py          # VerificationResult, registre type -> fonction
    common.py            # parsing indexé, comparaisons exactes, formatage
    inner_product.py  matrix.py  formula.py  value.py  property.py
    counterexample.py  equalities.py  dimensions.py
  hints.py  journal.py  lab.py  lock.py  panel.py
```

7.2 Tâches

N°	Tâche	Détail
3.1	Registre des vérificateurs	Remplacer la cascade `if v_type in (...)` de `ExerciseSession.verify` par un dictionnaire `{type: fonction}` ; un nouveau type = un module + une ligne d'enregistrement.
3.2	Découper `verifiers.py`	Un module par vérificateur ; `_EXPECTED_FORMS` rejoint chaque module (attribut `forme_attendue`).
3.3	Découper `catalog.py`	`session.py` et `validation.py` ; le contrôle croisé devient une méthode par vérificateur (`cross_check(spec, data)`), ce qui supprime la seconde cascade de `if`.
3.4	`safe_eval.py`	Interpréteur sur `ast` acceptant : nombres, noms autorisés, opérateurs arithmétiques, comparaisons, appels à une liste blanche (`Vector`, `Matrix`, `sqrt`, `Rational`, méthodes `.dot`, `.norm`…), listes, tuples, dictionnaires. Remplace les dix `eval`.
3.5	Spécification des jumeaux	Le champ `jumeau` des fiches (`generateurs.moyenne_glissante(fenetre=2, n=5)`) est analysé par `safe_eval` avec seulement l'espace de noms des générateurs.
3.6	Tests	Tests dédiés à `safe_eval` (expressions refusées : attributs privés, `__import__`, lambda, compréhensions).

Critère de fin : aucun fichier du tuteur au-delà d'environ 350 lignes, plus aucun `eval`, diagnostics identiques (phase 0).

8. Phase 4 — Déterminant exact avec discussion de cas

8.1 Fonctionnalités

Élément	Description
`Determinant(A, *, method="elimination", assumptions=None)`	Procédure tracée. `method="elimination"` : triangulation sans dilatation par le moteur, $\det A = (-1)^{\text{échanges}} \prod$ pivots. `method="cofactors"` : développement suivant une ligne ou colonne (pédagogique, limité à $n \leq 4$), choix automatique de la ligne la plus creuse. `method="sarrus"` pour $n = 3$.
`DetResult`	`value` (expression factorisée), `factors`, `method`, `trace` ; `explain()`, `verify()` (comparaison avec un second calcul).
`Matrix.det(**kw)`	Méthode de commodité ; renvoie une expression (éventuellement paramétrée).

Élément	Description
Discussion de cas	Comme le déterminant est un polynôme en les paramètres, il n'y a pas besoin de séparer pendant le calcul si l'on ne normalise pas : on renvoie l'expression factorisée **et** une méthode `zeros()` qui donne, via l'oracle et `Condition`, les valeurs annulant le déterminant (`a = 1`, `a = −2`). Option `split=True` pour obtenir un `CaseSplit` « inversible / non inversible ».
Propriétés tracées	Effet des opérations élémentaires (échange : signe, dilatation : facteur, transvection : invariance), $\det(AB) = \det A \det B$, $\det A^T = \det A$, matrices triangulaires et par blocs.

8.2 Rendu et locales

Nouvelles clés `det.*` dans `render/locales/fr.yaml` (début, pivot, échange de signe, produit final, développement par cofacteurs, mineur). Rendu LaTeX du développement : $\det A = \sum_j (-1)^{i+j} a_{ij} M_{ij}$ avec les mineurs affichés.

8.3 Intégration

- `Matrix.inv()` : en cas de paramètres, s'appuyer sur `det().zeros()` pour annoncer les cas singuliers avant l'élimination (message plus clair).
- Tuteur : 4 nouvelles fiches (thème « Déterminants ») : déterminant 3×3 par Sarrus, déterminant d'une matrice triangulaire par blocs, valeurs d'un paramètre rendant une matrice singulière, effet des opérations élémentaires. Vérificateurs existants (`valeur`, `propriete`) ; ajout éventuel d'un type `ensemble_valeurs`.
- Cahier d'exercices : remplacer l'indice « le déterminant vaut… » de l'exercice 16 par un appel possible à `det()` dans le corrigé.
- Documentation : section dans le guide 03, entrée dans la référence API, exemples dans le tutoriel.

8.4 Tests

- Comparaison avec `sympy.Matrix.det()` sur 50 matrices aléatoires entières et rationnelles ($n \leq 5$) et sur des matrices paramétrées.
- Égalité des trois méthodes sur $n = 3$.
- Propriétés : multiplicativité, transposée, signe d'un échange.
- Cas limites : matrice nulle, 1×1, non carrée (`DimensionMismatchError`).

9. Phase 5 — Valeurs propres, espaces propres, diagonalisation

9.1 Fonctionnalités

Élément	Description
`CharacteristicPolynomial(A)`	$\chi_A(\lambda) = \det(A - \lambda I)$ calculé par le moteur avec un symbole réservé λ ; résultat factorisé sur $\mathbb{Q}$ (et sur $\mathbb{Q}$(paramètres)).
`Eigen(A, *, assumptions=None)`	Valeurs propres réelles avec multiplicités algébriques ; pour chacune, base de l'espace propre $\ker(A - \lambda I)$ via le moteur, multiplicité géométrique.
`EigenResult`	`char_poly`, `eigenvalues` (dict valeur → multiplicité), `eigenspaces` (dict valeur → tuple de vecteurs), `diagonalizable`, `P`, `D`, `non_real_part` (facteur sans racine réelle), `trace`.
`Matrix.eigenvalues()`, `Matrix.eigenvectors()`, `Matrix.diagonalize()`, `Matrix.char_poly()`	Méthodes de commodité.
Puissances	`Matrix.power(n)` : par diagonalisation si possible (PD^nP^{-1}, avec `n` symbolique), sinon par décomposition $\lambda I + N$ lorsque N est nilpotente (comme l'exercice 20 du cahier), sinon produits successifs pour `n` entier.

9.2 Choix mathématiques à trancher

1. **Corps de travail** : le package travaille sur $\mathbb{R}$. Proposition : ne renvoyer que les valeurs propres **réelles** et signaler explicitement la partie du polynôme sans racine réelle (« λ^2+1 n'a pas de racine réelle : A n'est pas diagonalisable sur $\mathbb{R}$ »). Une option `field="C"` pourra venir plus tard.
2. **Racines non explicites** : au-delà du degré 4, ou pour des polynômes irréductibles de degré 3–4 dont les formules sont illisibles, SymPy produit des `CRootOf`. Proposition : accepter les racines rationnelles et quadratiques explicites, et pour les autres renvoyer le facteur irréductible avec un message pédagogique, sans tenter de formules de Cardan.
3. **Paramètres** : la diagonalisabilité dépend souvent d'un paramètre (valeurs propres qui coïncident). Le moteur produit naturellement le `CaseSplit` lors du calcul des noyaux $\ker(A - \lambda I)$; la condition « deux racines égales » se détecte en factorisant le discriminant ou par égalité formelle des facteurs.

9.3 Pédagogie

- Trace en trois temps : polynôme caractéristique (avec la trace du déterminant), racines (factorisation expliquée), espaces propres (élimination tracée sur $A - \lambda I$).
- Diagnostics clairs : « multiplicité géométrique 1 < multiplicité algébrique 2 pour $\lambda = 3$: non diagonalisable ».
- Vérification : $AP = PD$ contrôlé et tracé.
- Tuteur : thème « Valeurs propres et diagonalisation », 5 fiches (polynôme caractéristique 2×2, espace propre, diagonalisabilité selon un paramètre, A^n par diagonalisation, chaîne de Markov à deux états). Nouveau vérificateur `couple_propre` (vérifie $Av = \lambda v$, $v \neq 0$) et `diagonalisation` (vérifie P inversible et $P^{-1}AP$ diagonale).
- Problèmes d'exploration débloqués : dynamique de populations (Leslie), chaînes de Markov à long terme, suites récurrentes linéaires (Fibonacci).

9.4 Tests

- Comparaison avec `sympy.Matrix.eigenvects()` sur des matrices à valeurs propres rationnelles.

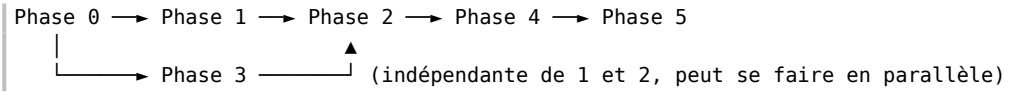
- Matrices de Jordan non diagonalisables, matrices symétriques (toujours diagonalisables), rotations (pas de valeur propre réelle).
- Matrices paramétrées : $\begin{bmatrix} 1 & a \\ 0 & 1 \end{bmatrix}$ (diagonalisable si et seulement si $a = 0$).
- A^n symbolique vérifié pour $n = 0, \dots, 8$.

10. Phase 6 — Extensions ultérieures

Classées par rapport intérêt / effort décroissant.

Piste	Description	Dépend de
Exercices paramétrés par graine	Données d'une fiche générées à partir d'une graine (numéro d'étudiant) ; les vérificateurs, qui contrôlent des propriétés, restent inchangés. Levier fort contre la copie.	phase 3
Sous-espaces comme objets	Classe `Subspace` : engendré, noyau, image ; dimension, base, somme, intersection, supplémentaire orthogonal, projection.	phase 1
Géométrie de $\mathbb{R}^2$ et $\mathbb{R}^3$	Produit vectoriel, droites et plans, intersections, distances, positions relatives avec discussion de cas, figures 3D.	phases 1 et 4
Factorisation LU (avec permutation)	Tracée comme Gauss-Jordan, reliée aux substitutions triangulaires.	phase 1
QR par Householder / Givens	Pour enrichir les comparaisons numériques.	—
Rejeu de trace dans JupyterLab	Curseur « étape suivante » sur les traces, pivot et lignes mis en évidence.	—
Générateur de sujets LaTeX/PDF	Feuilles d'exercices et sujets à variantes, avec corrigés.	exercices paramétrés
Parcours adaptatif et tableau de bord enseignant	Graphe de prérequis, agrégation des journaux d'une classe.	phase 3
Locale anglaise	Fichier `en.yaml`.	—
Publication PyPI et documentation en ligne	Après stabilisation de l'API.	phase 0

11. Calendrier et dépendances



Ordre	Phase	Séances	Jalon
1	Phase 0	1–2	CI verte, tests de caractérisation en place

Ordre	Phase	Séances	Jalon
2	Phase 1	2–3	`GaussJordan` reconstruit sur le moteur, sorties identiques
3	Phase 2	1	`Matrix` < 600 lignes, registre de procédures
4	Phase 3	2	tuteur découpé, plus d'`eval`
5	Phase 4	2–3	`det()` disponible, 4 fiches, documentation
6	Phase 5	3–4	`eigen()` et `diagonalize()`, 5 fiches, documentation — **version 3.1**

Total indicatif : **11 à 15 séances** jusqu'à la version 3.1.

12. Risques et parades

Risque	Probabilité	Impact	Parade
Régression invisible pendant les refactorisations	moyenne	fort	Tests de caractérisation (phase 0), commits atomiques, comparaison des carnets exécutés avant / après.
Explosion de la taille des expressions (paramètres, 4×4 et plus)	forte	moyen	Factorisation systématique, limite de taille documentée, message explicite au-delà.
Racines non explicites (degré ≥ 3)	forte	moyen	Politique claire (§ 9.2), facteurs irréductibles renvoyés tels quels.
Import circulaire lors du registre de procédures	faible	moyen	Registre alimenté par `algorithms/__init__.py`, comme le crochet actuel.
Synchronisation OneDrive gênant git ou l'écriture de fichiers	moyenne	faible	Réessayer les écritures ; si possible, travailler hors du dossier synchronisé.

13. Décisions à prendre

1. `requires-python` : `>=3.10` (recommandé) ou `>=3.11` ?
2. Corps des valeurs propres : $\mathbb{R}$ seul en 3.1 (recommandé) ou $\mathbb{C}$ en option dès le départ ?
3. Méthodes de déterminant exposées : élimination + cofacteurs + Sarrus (recommandé) ou élimination seule ?
4. Faut-il conserver les alias internes de l'ancien découpage du tuteur (`tutor.verifiers.verify_matrix`…) au-delà d'une version ?
5. Numérotation : la version 3.1 regroupe les phases 0 à 5 ; souhaites-tu des versions intermédiaires (3.0.1 après les refactorisations) ?

14. Définition de « terminé » (pour chaque étape)

- ☐ Tests unitaires et de caractérisation verts (`pytest`), carnets exécutés sans erreur.
- ☐ `validate_sheet` valide les 35 fiches (et les nouvelles).
- ☐ `ruff` sans erreur ; `mypy` sans nouvelle erreur.
- ☐ Documentation Markdown mise à jour, site HTML régénéré (`python build_docs_site.py`), liens vérifiés.

- ☐ `CHANGELOG.md` complété.
- ☐ Commit sur `development` avec un message explicite.

Annexe A — Point de reprise pour la prochaine séance

1. Commiter le travail en attente : `exercices/` (cahier de 20 exercices et corrigé) et le correctif `compute_inverse`, puis la documentation associée.
2. Installer les outils de la phase 0 : `pip install -e ".[dev]" coverage`.
3. Lancer `coverage run -m pytest && coverage report -m` et noter les modules sous 80 %.
4. Écrire les premiers tests de caractérisation des traces de `GaussJordan` (étape 0.2) : c'est le prérequis direct de la phase 1.

Annexe B — Esquisse de l'API cible (version 3.1)

```
from linalg_pedagopy import Matrix, symbols, Determinant, Eigen

a, = symbols("a")
A = Matrix([[a, 1, 1], [1, a, 1], [1, 1, a]])

A.det()                          # (a - 1)**2*(a + 2)
Determinant(A).run().zeros()     # a = 1, a = -2
Determinant(A, method="cofactors").run().explain(level=2)

B = Matrix([[2, 1], [1, 2]])
res = Eigen(B).run()
res.eigenvalues                  # {1: 1, 3: 1}
res.eigenspaces                  # {1: (Vector([-1, 1]),), 3: (Vector([1, 1]),)}
P, D = B.diagonalize()
n, = symbols("n")
B.power(n)                       # P D**n P**-1, en fonction de n

Matrix([[1, a], [0, 1]]).diagonalize()
# CaseSplit : a = 0 -> diagonalisable ; a ≠ 0 -> non diagonalisable (λ = 1 double, espace propre de
 ↪  dimension 1)
```